

并行算法理论作业2

陳日康 信息与计算科学 22336049

1. 使用 SIMD-CC 进行并行最大值查找 (25 分)

假设你有一个长度为 $n = 2^k$ 的整数数组 A ，其中第 i 个元素 $A[i]$ 由一台 SIMD-CC 并行机器中的一个处理器 p_i 存储。请设计一个算法，查找数组中最大的元素，并回答以下问题：

(1) 简要解释 SIMD-CC 并行机器模型。(5 分)

(2) 设计一个并行算法，在仅使用 SIMD-CC 支持的操作下，计算整个数组中的最大值，可以通过伪码对算法进行解释。(10 分)

(3) 分析你设计的算法的时间复杂度和总成本。你的算法是否是成本最优的？如果不是，是否可以修改算法以提高成本效率？(10 分)

解：

(1). SIMD-CC (Single Instruction, Multiple Data with Constant Communication) 是一种并行计算模型，它结合了单指令流与常数通信的特性。在该模型中，所有处理器在同一时刻由中央控制器统一调度，执行完全相同的指令，只不过操作的数据是各自本地私有的。这一特性被称为“单指令流，多数据流”(SIMD)。每个处理器拥有自己的局部数据存储单元，例如在本题中，数组 A 中的第 i 个元素 $A[i]$ 存储于编号为 i 的处理器 p_i 中。

除了执行同步操作外，SIMD-CC 模型还具有“常数通信”(Constant Communication) 的特点，即在每一轮计算中，每个处理器只能在常数时间内与固定数量的处理器通信。典型通信拓扑包括线性结构、环、树或网格等，通信配对关系往往由编号关系（如按位异或、位移）决定。这种通信模式既限制了复杂的任意通信，也简化了实现成本，非常适合数据高度结构化的并行问题。

(2).

```
// 每个处理器 p_i 执行以下代码
int A_i; // 本地存储的整数 A[i]
int i;   // 本地处理器编号

for (int d=0; d<log2(n); ++d) {
    int partner = i ^ (1 << d); // 计算当前轮次的通信伙伴编号

    // 向伙伴发送本地值，并接收其值
    int temp = send_and_receive(partner, A_i); // 模拟通信操作

    // 若本处理器编号小于对方编号，则保留较大值
    if (i<partner) {
        A_i = std::max(A_i, temp);
    }
}
```

```
// 否则，什么都不做（等待下一轮，或被淘汰）
}

// 最终，编号为 0 的处理器 p_0 拥有数组中的最大值
```

算法的每一轮中，处理器成对通信并比较局部值，在保持同步性的同时逐步向低编号集中最大值。因为通信仅在固定配对之间进行，且每轮中所有处理器并行执行，因此完全符合 SIMD-CC 的操作规范和通信限制。

(3). 该并行归约算法在 $\log_2 n$ 轮中完成查找，其中每一轮仅需常数时间完成通信与比较操作，因此整体时间复杂度为 $O(\log n)$ 。由于整个过程中有 n 个处理器参与执行，每轮都有活动处理器，因此总成本为处理器数乘以时间复杂度，即 $C(n) = O(n \log n)$ 。

从成本最优性的角度来看，串行最大值查找只需 $O(n)$ 时间即可完成，因此最优总成本应为 $O(n)$ 。相比之下，该并行算法的总成本高出一个 $\log n$ 的因子，因而不是成本最优的。虽然它在时间上获得了加速，但以牺牲大量处理器参与为代价，效率仍存在改进空间。

为了进一步提升成本效率，可以考虑将处理器划分为 $\sqrt{n}$ 组，每组内使用 $\sqrt{n}$ 个处理器并行查找组内最大值，然后再使用 $\sqrt{n}$ 个处理器对这些组内最大值进行归约。这样，两阶段各耗时 $O(\log \sqrt{n})$ ，总时间仍为 $O(\log n)$ ，但所需处理器总数从 n 降为 $O(\sqrt{n})$ ，从而将总成本降低为 $O(\sqrt{n} \log n)$ ，更接近成本最优的 $O(n)$ 。

因此，当前的算法虽然结构清晰、易于实现，有良好的并行性能，但其在处理器资源使用上的效率尚可优化。通过分组策略降低参与处理器数，能够有效压缩计算资源，提高整个系统的成本效益，同时保持时间复杂度不变。

2. 使用 SIMD-CRCW (Arbitrary Write) 进行直方图计算 (25 分)

假设你有一个长度为 n 的整数数组 $A[0, \dots, n-1]$ ，其中每个元素 $A[i]$ 的取值范围是 $[0, m-1]$ 。请使用 SIMD-CRCW (arbitrary write) 并行机器，计算该数组的直方图 $H[0, \dots, k-1]$ ，并回答以下问题：

(1) 简要解释 SIMD-CRCW 模型及 arbitrary write 的含义。说明该模型与 CREW、EREW 有哪些不同？(5 分)

(2) 请设计一个并行算法，在不使用额外的同步操作（如原子操作、临界区）的前提下，使用 SIMD-CRCW (arbitrary write) 模型并行计算直方图 H 。(10 分)

(3) 分析你所设计算法时间复杂度和成本的期望值，并讨论其与同步操作的优劣 (10 分)

解：

(1). SIMD-CRCW 是一种并行计算模型，其中 SIMD 表示单指令流、多数据流，即所有处理器在同一时刻执行相同的指令，但作用于不同的数据。CRCW 表示在该模型中，多个处理器可以在同一时刻对共享内存的同一地址进行写入操作，即“Concurrent Read, Concurrent Write”。这是对更严格模型如 CREW (Concurrent Read, Exclusive Write) 或 EREW (Exclusive Read, Exclusive Write) 的放宽。

在 CRCW 模型中，存在不同的并发写入规则，arbitrary write 是指如果多个处理器在同一时刻写入同一内存地址，最终写入的值是任意一个处理器写入的值。该规则并不指定优先级或冲突解决机制，因此处理器之间无需协调谁来写，写入结果是未定的，但保证不会出错或产生竞争异常。

而 CREW 模型允许多个处理器同时读取同一地址，但写入必须独占；EREW 模型更为严格，既不允许多个处理器同时读取同一地址，也不允许并发写入。因此，SIMD-CRCW 模型在并行能力上最强，允许最大程度的并发操作，尤其适合对共享内存结构不加协调地快速写入。

(2). 在 SIMD-CRCW (arbitrary write) 模型下，目标是并行计算数组 A 的直方图 H ，其中 A 的每个元素属于区间 $[0, k - 1]$ ，而 $H[x]$ 表示值为 x 的元素在 A 中出现的次数。由于 CRCW 允许多个处理器并发写同一地址，在 arbitrary write 模式下我们不能简单进行加法累加，否则会发生信息丢失。但可以通过如下策略避开原子加法等同步机制：

采用一个分布式方法：让每个处理器负责计算一个局部标志数组，然后在一个并行归约阶段对所有处理器的写入进行聚合。即每个处理器处理一个 $A[i]$ ，并在一张辅助布尔矩阵中设置 $H[i][A[i]] = 1$ 。然后，对每一列 $x = 0, \dots, k - 1$ ，统计第 i 行中是否存在 $H[i][x] = 1$ ，最后对列方向求和，得到最终的直方图 H 。

```
// 假设输入数组 A[0..n-1]，元素取值范围在 [0, k-1]
int A[n];           // 原始数据
int H[n][k];        // 局部直方图布尔矩阵
int FinalH[k];      // 输出直方图

// 每个处理器 p_i 并行执行以下操作
for (int i=0; i<n; ++i) {
    for (int j=0; j<k; ++j)
        H[i][j] = 0; // 初始化

    H[i][A[i]] = 1; // 将对应值位置设为 1
}

// 并行归约 (按列求和)
for (int j=0; j<k; ++j) {
    FinalH[j] = 0;

    // 对每一行累加 j 列的值
    for (int i=0; i<n; ++i)
        FinalH[j] += H[i][j];
}
```

上述伪码中的最后一个归约阶段可以使用并行归约操作完成，使整体时间复杂度保持在对数级别。该方法完全避开了同步操作（未用锁、原子加法或临界区），符合 SIMD-CRCW arbitrary write 的计算模型。

(3). 在该算法中，第一阶段每个处理器独立设置一个大小为 k 的布尔数组的一项 ($O(1)$ 时间)，因此可并行完成，耗时 $O(1)$ 。第二阶段中对每个直方图桶 j 进行 n 次累加，若使用树形归约，时间复杂度为 $O(\log n)$ 。由于有 k 个桶，该阶段可并行完成所有桶的求和，总体并行时间为 $O(\log n)$ 。

因此，整个算法的并行时间为： $T(n) = O(\log n)$ ，所需处理器数为： $P(n) = O(n)$ (n 个输入元素，每个处理器负责一个)。总成本为： $C(n) = T(n) \cdot P(n) = O(n \log n)$ 。

相比之下，串行直方图计算的时间复杂度为 $O(n)$ ，成本最优为 $O(n)$ 。因此，本算法在并行时间上获得了加速，但成本不是最优的。出现这一差距的原因在于：由于 arbitrary write 不支持累加操作，因此引入一个 $n \times k$ 的辅助布尔矩阵，从而增加了空间和归约的复杂度。

如果允许使用原子加法或互斥锁（即同步操作），可以直接令每个处理器对 `H[A[i]]++`，这样并行时间仍为 $O(1)$ ，成本为 $O(n)$ ，且空间仅为 $O(k)$ 。因此，在实际系统中引入同步机制反而可以获得更好的成本效率和资源利用率。

然而，在理论模型中，SIMD-CRCW (arbitrary write) 更强调算法在无同步条件下的正确性和可实现性。该算法的优点在于结构简单、完全并行，适用于处理器通信受限或模型中不允许原子操作的情况。但从总体资源效率来看，引入同步机制仍能带来更优的成本和空间效率，因此两者之间应视具体应用与硬件条件权衡取舍。

3. 使用并行合并实现分治的凸包计算 (20 分)

给定一个平面上 n 个点，假设你拥有一个并行函数，可以将两个不相交的凸包（每个大小为 n ）在 $O(\log n)$ 时间内，使用 n 个处理器合并为一个凸包。试通过该函数设计一个基于分治法的并行凸包计算算法，并分析其性能。

(1) 试描述该算法的思路及计算过程，可通过伪码进行说明 (10 分)

(2) 分析该算法的并行时间、所需处理器数量、及总成本，并说明该算法是否为成本最优？ (10 分)

解：

(1). 给定平面上 n 个点，将其不断划分为若干小规模子问题（即小点集的凸包），然后使用题中提供的“并行合并函数”将这些子凸包逐步合并，最终构造出全局凸包。

基本思路如下：首先将输入点集按照横坐标进行排序（或以任意固定顺序），然后递归地将其划分为左右两个子集，每个子集大小为 $\frac{n}{2}$ 。对子集分别递归计算凸包，最后使用并行合并函数将两个子凸包合并为一个新的凸包。由于题中已说明该合并操作可在 $O(\log n)$ 时间内使用 n 个处理器完成，因此整个合并过程在并行环境下可以非常高效地实现。实际上构成了一棵深度为 $\log n$ 的完全二叉树，叶节点为最小规模的点集（如单点或两个点），每个非叶节点对应一次凸包合并操作。

```
struct Point {
    double x, y;
};

struct ConvexHull {
    vector<Point> points;
};
```

```

// 合并两个凸包（已知提供的并行函数，时间  $O(\log n)$ ，用  $n$  个处理器）
ConvexHull MergeConvexHulls(const ConvexHull& left, const ConvexHull& right);

// 并行凸包分治算法
ConvexHull ParallelConvexHull(Point P[], int n) {
    if (n ≤ 1) {
        ConvexHull ch;
        if (n = 1)
            ch.points.push_back(P[0]);
        return ch;
    }

    // 将点集一分为二（假设按 x 坐标预排序）
    int mid = n / 2;
    ConvexHull leftHull, rightHull;

    // 模拟并行调用（真实并行执行需用线程库/OpenMP/TBB 实现）
    #pragma omp parallel sections
    {
        #pragma omp section
        {
            leftHull = ParallelConvexHull(P, mid);
        }
        #pragma omp section
        {
            rightHull = ParallelConvexHull(P + mid, n - mid);
        }
    }

    // 合并两个子凸包
    return MergeConvexHulls(leftHull, rightHull);
}

```

`MergeConvexHulls` 为题目中所描述的并行函数，可在 $O(\log n)$ 时间内合并两个大小为 n 的不相交凸包，需要保证每轮划分得到的点集是按照横坐标有序的，以确保凸包合并合法有效。

(2). 算法的时间复杂度分析基于递归分治结构。在每一层递归中，有 $O(n)$ 个点被划分为两个子问题，然后使用并行合并函数将两个凸包合并。由于每次合并使用 n 个处理器完成，合并操作耗时为 $O(\log n)$ 。递归的总层数为 $\log n$ 层（因为每次将问题规模减半），因此总的并行时间为：
 $T(n) = O(\log n) \cdot O(\log n) = O(\log^2 n)$ 。每一层最多使用 $O(n)$ 个处理器来并行进行凸包合并操作。因此，总体所需的处理器数量为： $P(n) = O(n)$ 。所以，总成本（即处理器数 $\times$ 并行时间）为：
 $C(n) = T(n) \cdot P(n) = O(n \log^2 n)$

而对于串行的 Graham 扫描或 Andrew 算法，其时间复杂度为 $O(n \log n)$ ，因此理论上的成本最优应为 $O(n \log n)$ 。对比之下，本算法的成本为 $(n \log^2 n)$ ，比最优略高一个 $\log n$ 因子，因此不是成本最优的。造成这种差距的主要原因在于每一层的凸包合并虽然用 n 个处理器压缩成 $O(\log n)$ 时间，但在层数达到 $\log n$ 时，造成总时间的平方级增长。

如果希望进一步优化成本，有两种思路：一是使用 work-efficient 的并行合并算法，每轮使用更少的处理器来降低资源消耗；二是在合并阶段逐步减少参与处理器数（比如通过中间层聚合策略），避免在每一层都使用满 n 个处理器。这些方法都可以在理论上将总成本压缩至 $O(n \log n)$ ，从而达到成本最优。

4. 使用并行归约实现并行排序及其在 SIMD 架构下的性能分析 (30 分)

给定一个长度为 n 的非负整数数组 $A[0, \dots, n-1]$ ，其中所有元素均在 $[0, 2^k]$ 范围内。请设计一个基于并行归约及 SIMD 计算模型的并行排序算法。

(1) 使用并行归约实现并行排序 (10 分)

- 描述如何使用并行归约实现排序算法中的关键步骤（无需实现并行规约）。
- 指出该排序方法是基于比较的，还是非比较的。

(2) 当 k 为常数时的性能分析 (10 分)

假设 k 是一个固定常数并且 SIMD 机器配备了 n 个处理器。

- 在此设定下，你设计的算法是否为成本最优？请分析其总工作量、并行深度与总成本，并判断其是否达到最优。
- 如果不是成本最优，请说明如何调整处理器数量以使其达到成本最优。

(3) 当 $k = O(\log n)$ 时的性能分析 (10 分)

假设 $k = O(\log n)$ ，如数组中元素取值范围为 $[0, n]$ ，且 SIMD 机器仍有 n 个处理器。

- 在此设定下，你的算法是否成本最优？请分析其工作量、并行深度与总成本。
- 如果不是成本最优，请说明如何调整处理器数量以使其达到成本最优。

解：

(1). 在 SIMD 架构下进行并行排序时，若已知数组元素均为非负整数，且取值范围为 $[0, 2^k]$ ，则可以设计一种基于非比较的排序算法，即并行计数排序。该算法关键在于通过并行归约操作统计出每个整数在数组中的出现次数，并结合前缀和计算将其分布到正确位置。

假设数组 A 有 n 个元素，首先创建一个大小为 $2^k + 1$ 的桶数组 H ，用于记录每个可能取值出现的频数。第一步中，每个处理器负责一个 $A[i]$ ，将其对应的桶 $H[A[i]]$ 的值设为 1。由于多个处理器可能同时写入同一个桶，这一阶段可通过并行归约对每个桶的位置执行加法运算，得到完整的频率统计 H 。

第二步中，利用并行前缀和（也是一种归约操作）计算桶数组 H 的前缀和 S ， $S[j]$ 表示所有小于 j 的值在排序数组中应出现的位置。这相当于确定了每个数值的起始写入位置。

最后，重新遍历 A 中的元素，将每个 $A[i]$ 写入位置 $S[A[i]]$ 并递增 $S[A[i]]$ （可通过辅助数组解决并发写入），从而得到排好序的新数组。

由于算法核心是基于计数与前缀和等操作，并不涉及元素之间的直接比较，因此它属于非比较排序算法，本质上是并行计数排序的变体。并行归约在其中起到了高效汇总计数和前缀求和的作用，是实现排序全流程的关键。

(2). 若 k 为常数，则数组 A 中的元素值域为 $[0, 2^k]$ ，其大小也是常数量级。此时直方图 H 和前缀和 S 的长度均为常数，计算它们的时间代价可视为 $O(1)$ 。

整个排序过程可分为三部分：统计频数、计算前缀和、填充排序结果。第一步中，每个处理器处理一个输入元素，仅需 $O(1)$ 时间即可完成对对应桶的计数。在 SIMD 模型中，允许使用并行归约对桶数组中所有位置进行求和操作，总时间复杂度为 $O(\log n)$ 。由于桶数量为常数，归约树的高度仍为 $O(\log n)$ 。第二步的前缀和归约过程与此类似，也为 $O(\log n)$ 。第三步中，每个处理器根据前缀和计算出的写入位置填入排序结果，时间复杂度仍为 $O(1)$ 。

因此，在 n 个处理器的 SIMD 模型下，该算法的并行运行时间为： $T(n) = O(\log n)$ ，工作总量为：每个处理器参与常数次操作，总工作量为： $W(n) = O(n)$ 。总成本为： $C(n) = T(n) \cdot P(n) = O(n \log n)$

与串行计数排序的 $O(n)$ 相比，该并行版本并非成本最优。为使其达到成本最优，可以将处理器数减少为： $P(n) = \frac{n}{\log n}$ 。这样， $T(n)$ 仍为 $O(\log n)$ ，而总成本变为 $O(n)$ ，达到成本最优。因此，在 k 为常数的情况下，为了达到最优成本，应控制处理器规模，使其与问题规模匹配，避免过度并行带来的冗余计算。

(3). 若 $k = O(\log n)$ ，则数组 A 的元素范围为 $[0, n]$ ，也即桶数组 H 和前缀和数组 S 的长度为 $O(n)$ 。这种情况下，直方图统计和前缀和求解不再是常数时间操作，而需要对 $O(n)$ 大小的数组执行并行归约。

在第一阶段中，每个处理器负责将 $A[i]$ 归入对应桶，桶数为 $O(n)$ ，可能发生多个处理器同时写入同一个桶位置，因此仍需执行 $O(\log n)$ 时间的归约（如二叉树求和）来统计频数。第二阶段中，前缀和的并行计算时间为 $O(\log n)$ 。第三阶段写入输出数组的过程，每个处理器根据前缀和结果决定元素位置，耗时为 $O(1)$ 。

因此，总的并行运行时间仍为： $T(n) = O(\log n)$ ，但由于桶数不再是常数，而是 $O(n)$ ，归约过程对每个桶都执行一次，导致工作总量增大至： $W(n) = O(n)$ 。处理器数仍为 $P(n) = \frac{n}{\log n}$ ，因此总成本为： $C(n) = O(n \log n)$ 。这与比较排序的下界一致，因此即使不成本最优，也与最优相差不大。但我们仍可以进一步优化处理器使用。若将处理器数量调整为： $P(n) = \frac{n}{\log n}$ 。则并行时间仍保持在 $O(\log n)$ ，而总成本变为： $C(n) = O(n)$ 。因此，在 $k = O(\log n)$ 的条件下，通过适度减少处理器数量，同样可以将该排序算法调整为成本最优。